



**Universitatea
Transilvania
din Brașov**

**FACULTATEA DE ȘTIINȚE ECONOMICE
ȘI ADMINISTRAREA AFACERILOR**

LUCRARE DE DISERTAȚIE

Conducător științific:

Conferențiar dr. BÎLDEA Teodor-Ștefan

Absolvent:

Bolboacă Mara

BRAȘOV, 2026

*Universitatea Transilvania din Braşov
Facultatea de Ştiinţe Economice şi Administrarea Afacerilor
Program de studii: Sisteme Informatice Integrate în Afaceri*

Optimizarea stocurilor în sectorul retail folosind metode predictive

Conducător ştiinţific:

Conferenţiar dr. BÎLDEA Teodor-Ştefan

Absolvent:

Bolboacă Mara

BRAŞOV, 2026

CUPRINS

REZUMAT

Scurtă descriere a lucrării.

Scopul lucrării.

Obiectivele lucrării.

Metode și instrumente utilizate.

Contribuția personală.

Cuvinte-cheie: 3-5 termeni.

CAPITOLUL 1 - Introducere

1.1 Contextul general

În contextul dezvoltării rapide a soluțiilor bazate pe inteligența artificială în tot mai multe domenii ale vieții economice și sociale, pentru a rămâne competitive și relevante pe piață, majoritatea companiilor își aliniază activitatea la noile cerințe tehnologice.

Dincolo de trend, inteligența artificială aduce îmbunătățiri incontestabile în activitatea organizațiilor prin automatizarea sarcinilor repetitive, reducerea timpilor de procesare, creșterea acurateții analizelor și îmbunătățirea capacității de prognoză.

„Cea mai bună cale de a prezice viitorul este să-l crezi.” – Peter Drucker¹

Prezenta lucrare se concentrează asupra metodelor de predicție construite cu inteligența artificială, folosite pentru a determina viitoare informații despre afaceri bazându-se pe tipare istorice. Valoarea principală a acestor metode, nu este „prezicerea viitorului”, ci identificarea și conștientizarea diferitelor scenarii posibile, fundamentate pe tendințe istorice reale, precum și dezvoltarea abilităților de adaptare în fața neprevăzutului. În acest sens, metodele predictive bazate pe inteligență artificială nu vizează exclusiv anticiparea unor valori viitoare, ci oferă un cadru analitic pentru înțelegerea și gestionarea incertitudinii.

Tema este fundamentată pe un studiu de caz real, realizat pe baza datelor operaționale ale unei organizații comerciale, în care procesele de aprovizionare, vânzare și gestiune a stocurilor sunt susținute de un sistem ERP conectat la o bază de date relațională.

Problema cercetată

În urma analizei capabilităților și nevoilor companiei în care lucrez, una dintre provocările întâlnite de clienți o reprezintă planificarea eficientă a stocurilor, provocare des discutată în cadrul sistemelor informatice integrate din retail în condițiile unui comportament de consum variabil și al unor termene de livrare dependente de furnizori. În practica organizației analizate, deciziile de aprovizionare se bazează în principal pe indicatori simpli, precum vânzarea medie zilnică, utilizată pentru estimarea cantităților comandate și a perioadei de acoperire a stocului.

Deși această abordare oferă un nivel de control operațional, ea nu integrează în mod explicit dinamica cererii și nu permite o anticipare suficient de precisă a situațiilor de suprastoc sau de lipsă de stoc. Astfel, comenzile sunt planificate pe baza unui număr fix de zile de aprovizionare, stabilit fie la nivel de furnizor, fie printr-o variabilă generală, fără a ține cont de evoluția reală a cererii sau de variațiile istorice ale vânzărilor.

În plus, lipsa unor estimări pe termen mediu sau a unei integrări a evenimentelor sezoniere conduce la decizii reactive, cu impact direct asupra performanței lanțului de aprovizionare. Probleme suplimentare apar și din calitatea datelor operaționale, cum ar fi situațiile de stoc negativ, erorile de recepție sau scanare și perioadele de „out of stock”, care pot distorsiona analiza dacă nu sunt tratate corespunzător.

Importanța problemei

Gestionarea ineficientă a stocurilor are un impact semnificativ asupra performanței financiare și operaționale a organizațiilor din retail. Suprastocarea conduce la blocarea capitalului, creșterea costurilor de depozitare și riscul de depreciere a produselor, în timp ce situațiile de „out of stock” generează pierderi de vânzări, scăderea satisfacției clienților și afectarea imaginii comerciale.

În contextul analizat, optimizarea stocurilor nu reprezintă doar o problemă de reducere a costurilor, ci un element strategic pentru susținerea competitivității. Integrarea prognozelor generate prin metode predictive în procesele ERP permite trecerea de la o abordare bazată

¹ StartCo - Newsletter pentru antreprenori

pe vânzări istorice simple la una orientată către cererea prognozată, corelată cu timpii reali de aprovizionare și cu nivelurile minime de stoc dorite.

1.2 Obiectivele cercetării

Obiectivul principal al lucrării este analizarea și evaluarea modului în care metodele predictive bazate pe inteligență artificială pot fi utilizate pentru optimizarea proceselor de gestiune a stocurilor, pornind de la datele operaționale extrase dintr-un sistem informatic ERP și valorificate ulterior prin instrumente de Business Intelligence. În acest context, sistemul ERP este utilizat ca sursă de date, fără a fi modificat din punct de vedere funcțional, iar accentul este pus pe analiza și interpretarea rezultatelor predictive în stratul analitic.

Lucrarea urmărește să evidențieze impactul utilizării prognozelor de cerere generate pe baza datelor istorice asupra procesului de planificare a comenzilor către furnizori, comparativ cu abordarea tradițională bazată pe vânzarea medie zilnică. De asemenea, este analizat un flux automatizat de prognoză realizat în Python, care prelucrează datele operaționale și furnizează rezultate structurate pentru analiză și vizualizare în instrumente de Business Intelligence. Prin această abordare, se urmărește identificarea diferențelor dintre cele două metode de fundamentare a deciziilor de aprovizionare, în special în ceea ce privește nivelurile de stoc rezultate și capacitatea organizației de a reduce situațiile de suprastoc sau lipsă de stoc.

1.3 Contribuția lucrării

Contribuția acestei lucrări constă în propunerea și analizarea unei soluții analitice pentru optimizarea gestiunii stocurilor, bazată pe utilizarea metodelor predictive de inteligență artificială aplicate pe date reale dintr-un sistem ERP. Lucrarea evidențiază modul în care datele operaționale pot fi valorificate într-un strat analitic independent, fără a interveni asupra logicii funcționale a sistemului ERP, prin intermediul unui flux automatizat de prelucrare și prognoză realizat în Python.

Prin studiul de caz realizat, lucrarea contribuie la reducerea decalajului dintre modelele teoretice de prognoză prezentate în literatura de specialitate și aplicabilitatea lor practică în organizații comerciale. Totodată, sunt evidențiate bune practici privind utilizarea instrumentelor de Business Intelligence ca suport decizional în procesele de optimizare a stocurilor, oferind un cadru replicabil pentru organizații care doresc să îmbunătățească performanța operațională prin analitică predictivă.

Capitolul 2. Stadiul actual al cercetărilor și aplicațiilor în domeniu (5–7 pagini)

2.1. Rolul sistemelor informatice integrate (ERP) în gestiunea și optimizarea stocurilor

Sisteme ERP – Definiție și rol în retail

Sistemele pentru planificarea resurselor întreprinderii (Enterprise Resource Planning – ERP) reprezintă platforme software integrate care centralizează și automatizează procesele de afaceri ale unei organizații, de la aprovizionare și vânzări până la gestiunea stocurilor și raportare financiară. Aceste sisteme au evoluat de la simple instrumente de management al producției către soluții comprehensive care integrează întreaga activitate operațională într-o singură bază de date centralizată ².

Sistemele ERP permit monitorizarea continuă a nivelurilor de stoc, a mișcărilor de intrare/ieșire și a valorii inventarului pe magazine, depozite sau locații multiple. Această vizibilitate elimină problemele generate de datele fragmentate și permite identificarea rapidă a anomaliilor ³.

Limitări și nevoia de instrumente complementare

Conform EazyStock (2025), multe module de gestiune a stocurilor din ERP oferă funcționalități de prognoză a cererii, de obicei bazate pe calcule simple de medie mobilă liniară, dar aceste metode nu pot lua în considerare cauzele tipice ale volatilității cererii.

Spre deosebire de metodele tradiționale, un sistem de optimizare a stocurilor va lua în considerare ciclul de viață al produsului, sezonabilitatea, tendințele și informațiile despre promoții – aspecte pe care modulele standard ERP nu le gestionează eficient. Această abordare complexă este ilustrată în figura de mai jos (fig. 1.), care evidențiază modul în care prognozele avansate integrează atât prognoza de bază (base forecast), cât și factori calitativi suplimentari precum tendințele (trends), sezonabilitatea (seasonality factors) și alte variabile contextuale (qualitative input).⁴

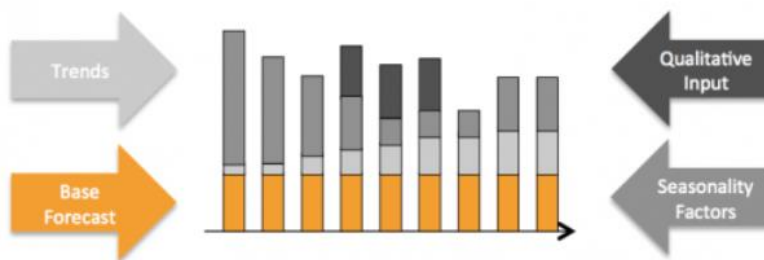


Fig. 1 Factorii care afectează prognoza cererii

2.2. Utilizarea Data Science și Machine Learning în prognoza cererii

Data Science permite analizarea volumelor mari de date istorice provenite din sisteme informatice integrate și identificarea unor tipare complexe de consum, oferind o bază mai solidă pentru estimările predictive comparativ cu metodele tradiționale. În special în sectorul de retail, caracterizat prin sezonabilitate, promoții frecvente și variații ale comportamentului consumatorilor, aceste tehnici contribuie la o mai bună înțelegere a dinamicii cererii și la fundamentarea deciziilor operaționale ⁵.

² <https://www.oracle.com/erp/what-is-erp/>

³ SAP, 2024 - <https://www.sap.com/products/erp/what-is-erp.html>

⁴ <https://www.eazystock.com/blog/erp-inventory-management-how-advanced-is-your-erp/>

⁵ <https://www.ibm.com/think/topics/ai-demand-forecasting>

Un aspect evidențiat în cercetarile recente este faptul că valoarea reală a inteligenței artificiale în prognoza cererii nu constă exclusiv în creșterea acurateței estimărilor, ci în capacitatea de a gestiona variabilitatea și incertitudinea inerente proceselor operaționale. Modelele predictive sunt utilizate pentru a analiza scenarii alternative și pentru a evalua impactul deciziilor asupra fluxului de marfă și asupra capitalului blocat în stocuri, mutând accentul de la o prognoză punctuală către înțelegerea comportamentului sistemului în ansamblu.⁶

Ce este Machine Learning?

Machine Learning este ramura inteligenței artificiale ce cuprinde dezvoltarea de algoritmi capabili să învețe automat din date și să își îmbunătățească performanța fără să fie programați detaliat pentru fiecare situație. Prin identificarea tiparelor și a relațiilor din seturi mari de date, aceste algoritmi pot realiza predicții sau pot sprijini luarea deciziilor în contexte complexe, precum prognoza cererii, analiza comportamentului consumatorilor sau optimizarea proceselor operaționale. Machine Learning este utilizat pe scară largă în mediile de afaceri pentru a transforma datele istorice în informații relevante, cu rol de suport decizional într-un mediu economic dinamic și competitiv.⁷

2.2.2. Tipuri de abordări în Machine Learning utilizate în prognoza cererii

Machine Learning poate fi organizat în mai multe paradigme care diferă în funcție de natura datelor și de modul în care acestea sunt utilizate pentru învățare. (Fig.2.)

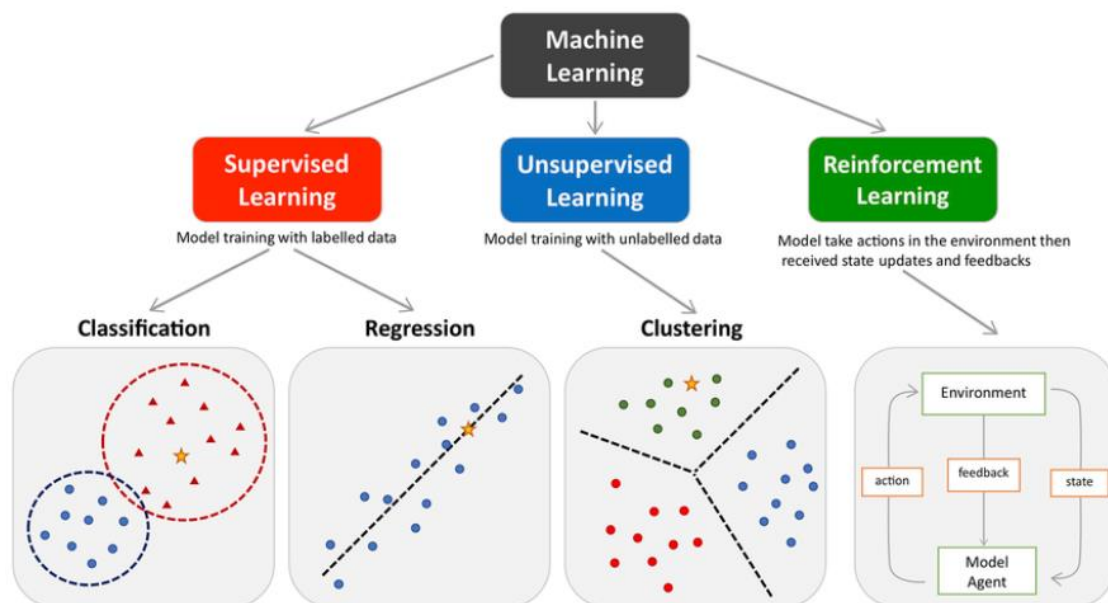


Fig. 2 Principalele tipuri de machine learning

⁶ <https://throughput.world/blog/ai-in-retail-supply-chain/>).

⁷ <https://www.ibm.com/think/topics/machine-learning>

Învățarea supervizată (supervised learning) este cea mai utilizată abordare în prognoza seriilor temporale atunci când scopul este să se prezică valori viitoare pe baza unor date istorice etichetate. În acest caz, datele de intrare și ieșire sunt cunoscute, iar modelul învață relația dintre variabile folosind tehnici de regresie sau alte algoritmi de predicție. Această paradigmă este esențială pentru modele care transformă datele secvențiale în exemple de predicție, cum ar fi regresia sau rețelele neuronale folosite pentru a estima cererea viitoare pe baza observațiilor trecute.⁸

In contextul algoritmilor exista variabile dependente / target / date etichetate si separarea setului de date in subseturi de antrenare si testare pentru evaluarea performantei modelului. Exemple de algoritmi : Regresie liniara, Regresie Logistica, k-NN (k-Nearest Neighbors), SVM (Support Vector Machines), Decision Tree, Gradient Boosting (XGBoost, LightGBM, CatBoost).

Pe de altă parte, **învățarea nesupervizată (unsupervised learning)** implică modele care explorează structura internă a datelor fără a avea o etichetă ca rezultat. În contextul gestiunii lanțului de aprovizionare, acest tip de abordare poate fi folosit pentru **identificarea de tipare ascunse, gruparea produselor** cu comportamente de vânzare similare sau **detectarea de anomalii** în datele de vânzări sau de inventar. Algoritmi precum K-means sau analiza principală de componente (PCA) pot ajuta la descoperirea acestor tipare fără a avea o ieșire clar definită de predicție.⁹

În cazul algoritmilor de învățare nesupervizată, nu există o variabilă dependentă sau date etichetate, iar analiza se realizează asupra întregului set de date cu scopul de a identifica structuri, tipare sau grupări interne, evaluarea rezultatelor bazându-se pe metrici interne și nu pe separarea clasică în seturi de antrenare și testare.

Încă o paradigmă este **învățarea prin recompensă (reinforcement learning)**, care, spre deosebire de primele două, nu se concentrează pe predicția directă a valorilor viitoare, ci pe învățarea unei politici optime de acțiune într-un mediu dinamic. Deși nu este folosită în mod obișnuit pentru prognoza pură a cererii, cercetările au explorat aplicarea sa în decizii operaționale cum ar fi planificarea comenzilor într-un lanț de aprovizionare sau controlul inventarului în scenarii dinamice. Astfel de abordări presupun interacțiunea cu un mediu ce oferă recompense sau penalizări pe baza acțiunilor întreprinse, ceea ce poate fi relevant pentru optimizarea deciziilor pe termen lung în gestionarea stocurilor.¹⁰

Reinforcement learning este un cadru de învățare în care un agent învață prin interacțiune cu mediul să ia decizii secvențiale, maximizând o recompensă cumulativă pe termen lung. Exemple de algoritmi RL: Q-Learning, Deep Q-Network, Actor–Critic Proximal Policy Optimization .

2.2.3. Serii temporale în retail

⁸ <https://www.geeksforgeeks.org/machine-learning/time-series-forecasting-as-supervised-learning/>

⁹ [Unsupervised Learning in Supply Chain Demand - growth-onomics](#)

¹⁰ <https://doi.org/10.1016/j.tre.2020.101926>

Seriile temporale sunt definite ca observații ordonate cronologic, utilizate pentru analizarea și prognozarea evoluției unei variabile în timp. În analiza cererii, seriile temporale sunt decompozabile în componente distincte, dintre care cercetarile de specialitate identifică explicit trendul, sezonabilitatea și componenta aleatorie.¹¹

Sezonabilitatea este descrisă ca o variație sistematică și repetitivă care apare la intervale regulate într-o serie temporală. Această componentă este asociată cu efecte calendaristice recurente, cum ar fi luni, trimestre sau zile ale săptămânii, și este prezentă atunci când aceleași tipare se repetă în aceeași perioadă a fiecărui ciclu. Definiția și caracteristicile sezonabilității sunt prezentate explicit în Hyndman și Athanasopoulos (2021, Secțiunea 2.3 – “Times series patterns”).

Trendul reprezintă direcția generală de evoluție a unei serii temporale pe termen lung, manifestată printr-o creștere sau o scădere persistentă a nivelului mediu al observațiilor. Trendul reflectă schimbări structurale lente ale procesului generator de date și este distinct de fluctuațiile sezoniere sau neregulate.¹²

În contextul cererii din retail, seriile temporale pot prezenta un comportament particular cunoscut sub denumirea de cerere intermitentă. *Cererea intermitentă* este caracterizată prin apariția frecventă a valorilor zero, urmate de observații nenule care apar la intervale neregulate.

Syntetos și Boylan (2005) descriu cererea intermitentă ca având două dimensiuni distincte: incertitudinea momentului apariției cererii și variabilitatea cantității solicitate atunci când cererea se manifestă. Aceste caracteristici fac ca seriile intermitente să fie dificil de modelat folosind metode standard de prognoză pentru serii temporale continue.

Literatura indică faptul că prezența unui număr mare de observații nule limitează capacitatea modelelor clasice de a identifica corect structuri precum trendul sau sezonabilitatea la nivelul seriei individuale.¹³

2.2.4. Metode statistice vs Machine Learning

În literatura de specialitate orientată spre practică, *statistical learning* și *machine learning* sunt prezentate ca domenii apropiate, dar cu obiective și accente diferite. În timp ce *statistical learning* pune accent pe explicarea și interpretarea relațiilor dintre variabilele dintr-un set de date, *machine learning* este descris ca fiind centrat pe predicție, clasificare și pe capacitatea algoritmilor de a „învăța” din date.

Statistical learning (analiză statistică). Statistical learning este definit ca o ramură din data science care urmărește să înțeleagă modul în care variabilele (sau seturile de date) se relaționează în interiorul unui set de date și oferă bază teoretică pentru multe tehnici predictive și analitice, inclusiv pentru unele metode utilizate în machine learning. În această abordare, analiza pornește frecvent de la o ipoteză privind structura datelor, iar apoi sunt

¹¹ Hyndman & Athanasopoulos, 2021, **Capitolul 3 – “Time series decomposition”**

<https://otexts.com/fpp3/decomposition.html>

¹² <https://otexts.com/fpp3/tspatterns.html>

¹³ <https://www.sciencedirect.com/science/article/abs/pii/S0169207004000792?via%3Dihub>

construite modele statistice pentru a descrie tipare și pentru a realiza predicții în acord cu aceste tipare.

Analiza statistică este utilizată pentru testarea ipotezelor, explorarea datelor atunci când rezultatele nu sunt încă evidente, modelarea relațiilor dintre variabile (ex. regresie, corelații, ANOVA) și identificarea de tendințe și tipare care pot susține predicții viitoare. În privința avantajelor, sunt menționate scalabilitatea la seturi de date de dimensiuni diferite, versatilitatea pe tipuri variate de date (continue, categoriale, serii temporale) și aplicabilitatea metodelor în contexte diferite. Ca limitări, sunt evidențiate dependența de date istorice (și presupunerea că tiparele se mențin) și existența unor ipoteze statistice stricte (de ex., independența variabilelor, distribuții normale), a căror încălcare poate complica analiza și poate reduce fidelitatea reprezentării datelor.

Machine learning. Machine learning este descris ca subdomeniu al inteligenței artificiale care urmărește dezvoltarea de algoritmi capabili să învețe din date și să-și îmbunătățească performanța în timp, fără instrucțiuni explicite pentru fiecare pas. În comparație cu statistical learning, accentul este pus pe identificarea de tipare (inclusiv unele dificil de observat direct) în seturi mari și complexe și pe obținerea de predicții pentru rezultate sau clasificări.

Modelele ML pot lucra cu volume mari de date, inclusiv date ne-structurate (ex. text, imagini), precum și posibilitatea de a se adapta la informație nouă și la aplicații diferite. Ca dezavantaje, este evidențiată dificultatea de a explica transparent procesul intern al unor algoritmi (ceea ce poate îngreuna identificarea erorilor sau justificarea deciziilor) și dependența performanței de calitatea datelor de antrenare; seturile mici sau dezechilibrate pot introduce bias și pot limita generalizarea.

Criterii de alegere între cele două abordări. Sursa recomandă evaluarea: (1) **complexității datelor** (datele slab structurate sau cu dimensionalitate ridicată pot favoriza ML), (2) **nevoii de explicabilitate** (modelele statistice pot oferi mai multă transparentă a pașilor de decizie) și (3) **resurselor disponibile** (modelele ML pot solicita resurse computaționale și expertiză, în timp ce modelele statistice pot necesita mai puține resurse, dar presupun respectarea anumitor ipoteze). ¹⁴

2.2.5. Modele utilizate în literatura de specialitate

ARIMA (AutoRegressive Integrated Moving Average)

Modelul ARIMA este un model statistic clasic utilizat pentru analiza și prognoza seriilor temporale univariate, fiind fundamentat pe metodologia Box–Jenkins. Denumirea **ARIMA** provine din combinarea termenilor *AutoRegressive* (AR), *Integrated* (I) și *Moving Average* (MA) și reflectă structura matematică a modelului. Modelul este definit prin parametrii (**p, d, q**),

¹⁴ [Statistical Learning vs. Machine Learning: What's the Difference? | Coursera](#)

care controlează modul în care valorile trecute ale seriei și erorile asociate sunt utilizate în estimarea valorilor viitoare.¹⁵

Componenta autoregresivă, de ordin **p**, descrie relația liniară dintre valoarea curentă a seriei și un număr finit de observații anterioare. Această componentă permite captarea dependenței temporale directe dintre observațiile succesive. Parametrul **d** reprezintă gradul de diferențiere aplicat seriei pentru a elimina non-staționaritatea, fiind esențial pentru stabilizarea mediei și a varianței în timp. Componenta de medie mobilă, de ordin **q**, modelează influența termenilor de eroare din perioadele anterioare asupra valorii curente.¹⁶

Modelul ARIMA este aplicabil în situațiile în care seria temporală nu prezintă sezonalitate sau în care efectele sezoniere au fost eliminate prin preprocesare. În documentațiile tehnice, se subliniază faptul că performanța modelului depinde de identificarea corectă a ordinilor **p**, **d** și **q**, pe baza analizei autocorelației și autocorelației parțiale.

Modelul ARIMA realizează predicțiile pe baza valorilor anterioare ale seriei temporale și a erorilor de estimare din trecut. Într-o manieră intuitivă, acesta poate fi privit ca un mecanism care combină informația recentă privind evoluția cererii cu corecții bazate pe abaterile anterioare ale modelului. Spre deosebire de modelele bazate pe rețele neuronale, ARIMA nu învață tipare complexe, ci se concentrează pe relațiile locale dintre observațiile succesive, ceea ce îl face adecvat pentru serii relativ stabile, fără trend pronunțat.

Pot apărea limitări în situațiile în care datele sunt puternic sezoniere sau au un comportament neliniar, cazuri în care acuratețea prognozelor poate scădea.

SARIMA (Seasonal AutoRegressive Integrated Moving Average)

Pentru seriile temporale care prezintă un comportament sezonier regulat, modelul ARIMA este extins la forma **SARIMA** (*Seasonal AutoRegressive Integrated Moving Average*). Acest model integrează explicit sezonalitatea prin introducerea unui set suplimentar de parametri sezonieri și este notat sub forma **ARIMA(p, d, q) × (P, D, Q, s)**.

Parametrii **P**, **D**, **Q** reprezintă analogii sezonieri ai componentelor nesezoniere:

- **P** indică ordinul componentei autoregresive sezoniere (*Seasonal AR*), care modelează relația dintre observații separate printr-un ciclu sezonier complet,
- **D** reprezintă gradul de diferențiere sezonieră (*Seasonal Integrated*), utilizat pentru eliminarea tendințelor sezoniere persistente,
- **Q** indică ordinul componentei de medie mobilă sezoniere (*Seasonal MA*), asociată termenilor de eroare la nivel sezonier.

Parametrul **s** (denumit frecvent și **m**) reprezintă perioada sezonalității și corespunde numărului de observații care formează un ciclu complet. De exemplu, pentru date lunare cu sezonalitate anuală, valoarea lui **s** este 12.¹⁷

¹⁵ <https://www.statsmodels.org/stable/generated/statsmodels.tsa.arima.model.ARIMA.html>

¹⁶ <https://otexts.com/fpp2/arima.html>

¹⁷ <https://otexts.com/fpp2/seasonal-arima.html>

În această notare:

- **SAR** provine din *Seasonal AutoRegressive*,
- **I** și **D** se referă la operațiile de integrare (diferențiere),
- **MA** desemnează componenta de medie mobilă,
- **M** este utilizat pentru a indica frecvența sezonieră a seriei.

Modelul SARIMA permite modelarea simultană a dependențelor pe termen scurt și a structurii sezoniere recurente, fiind utilizat frecvent în aplicații de prognoză economică și operațională.

Exponential Smoothing (Holt–Winters)

Metodele de *exponential smoothing* constituie o clasă distinctă de modele statistice pentru prognoza seriilor temporale, bazate pe ideea de ponderare exponențială a observațiilor trecute. Metoda Holt–Winters extinde netezirea exponențială simplă prin includerea explicită a unei componente de trend și a unei componente sezoniere.¹⁸

În această abordare, valorile recente ale seriei primesc o pondere mai mare decât observațiile mai vechi, iar nivelul, trendul și sezonalitatea sunt actualizate recursiv la fiecare pas temporal. Documentația statsmodels precizează că metoda poate fi utilizată cu sezonaliitate aditivă sau multiplicativă, în funcție de modul în care amplitudinea fluctuațiilor sezoniere variază în timp.

Holt–Winters este frecvent utilizată ca metodă de referință în studiile comparative de prognoză, datorită simplității sale și a capacității de a modela direct structuri sezoniere regulate.

Modele de Machine Learning pentru prognoza seriilor temporale

XGBoost (Extreme Gradient Boosting)

XGBoost este un algoritm de *gradient boosting* bazat pe arbori de decizie, conceput pentru a optimiza performanța și eficiența în probleme de regresie și clasificare. Documentația oficială descrie XGBoost ca un cadru de învățare care combină mai mulți arbori slabi într-un model puternic, prin minimizarea iterativă a unei funcții obiectiv .

În prognoza seriilor temporale, XGBoost este utilizat prin reformularea problemei într-un cadru de învățare supravegheată. Aceasta presupune construirea unui set de variabile explicative derivate din istoricul seriei, precum valori întârziate (*lag features*) și caracteristici calendaristice. Modelul nu presupune staționaritatea seriei și poate surprinde relații neliniare între variabilele explicative și variabila țintă .¹⁹

¹⁸ <https://www.statsmodels.org/stable/generated/statsmodels.tsa.holtwinters.ExponentialSmoothing.html>

¹⁹ <https://xgboost.readthedocs.io/en/stable/>

DE ADAUGAT MAI MULTE INFORMATII – CONCENTRARE PE XGBOOST DACA IL FOLOSESC IN APLICATIE**Random Forest Regression**

Random Forest este un algoritm de tip *ensemble learning* care utilizează un ansamblu de arbori de decizie antrenați pe eșantioane bootstrap ale datelor. Predicția finală este obținută prin agregarea (de regulă media) predicțiilor individuale ale arborilor. Această abordare reduce varianța modelului și îmbunătățește stabilitatea rezultatelor .

În aplicațiile de prognoză a seriilor temporale, Random Forest este utilizat într-un mod similar cu XGBoost, prin transformarea seriei într-un set de date supravegheat. Documentația subliniază faptul că modelul nu impune ipoteze stricte privind distribuția datelor sau structura temporală, fiind astfel potrivit pentru serii cu comportament neliniar. ²⁰

LSTM (Long Short-Term Memory)

Modelul **Long Short-Term Memory (LSTM)** este o extensie a rețelelor neuronale recurente (RNN), concepută pentru a depăși limitările acestora în modelarea seriilor temporale. Rețelele RNN clasice întâmpină dificultăți în reținerea informației relevante pe intervale lungi de timp, fenomen cunoscut sub denumirea de *vanishing gradient*. LSTM rezolvă această problemă prin introducerea unui mecanism explicit de memorie. LSTM este o arhitectură de rețele neuronale recurente concepută pentru modelarea datelor secvențiale și a dependențelor pe termen lung. Documentația TensorFlow descrie LSTM ca o extensie a RNN-urilor clasice, care utilizează mecanisme de tip *input gate*, *forget gate* și *output gate* pentru a controla fluxul informației în timp . ²¹

În prognoza seriilor temporale, LSTM este utilizat pentru a învăța relații temporale complexe direct din date, fără a necesita specificarea explicită a componentelor de trend sau sezonaliitate. Această capacitate face ca modelele LSTM să fie frecvent investigate în studii recente de prognoză bazată pe inteligență artificială.

Principiul de funcționare

Arhitectura LSTM este construită în jurul unei **celule de memorie**, care permite stocarea informației pe termen lung. Actualizarea acestei memorii este controlată de trei mecanisme denumite **porți (gates)**, care reglează fluxul de informație în rețea:

- **Forget gate** – decide ce informații din starea anterioară sunt eliminate;
- **Input gate** – stabilește ce informații noi sunt adăugate în memoria celulei;
- **Output gate** – determină ce parte din informația memorată este utilizată pentru generarea ieșirii.

²⁰ <https://scikit-learn.org/stable/modules/generated/sklearn.ensemble.RandomForestRegressor.html>

²¹

Prin acest mecanism, LSTM poate păstra informația relevantă și elimina zgomotul, adaptându-se dinamic la evoluția seriei temporale.²²

2.3. Aplicații practice și rezultate raportate în literatura de specialitate

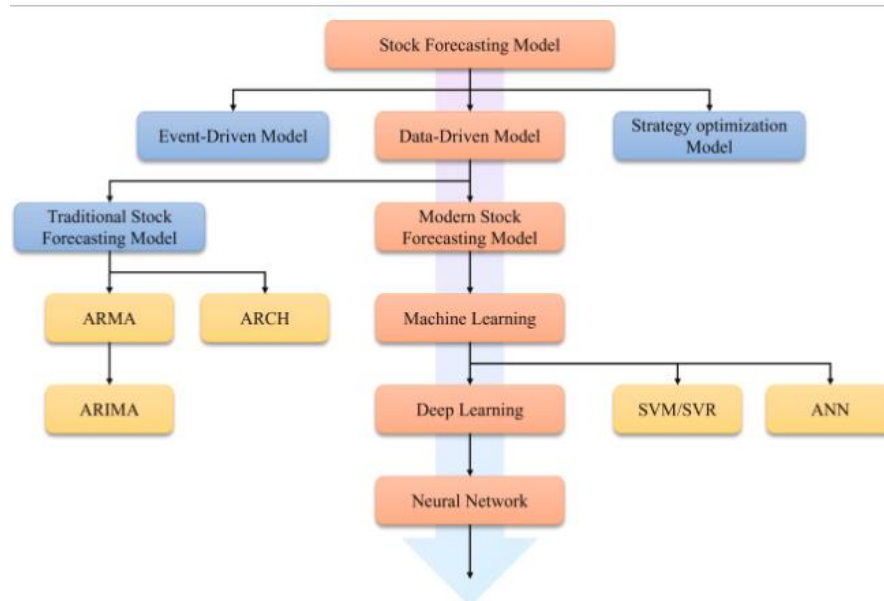


Fig. 3 Schema prognozelor studiate în lucrarea referita

-
- Studii de caz internaționale
- Beneficii raportate (reducere rupturi de stoc, optimizare costuri)

Potrivit unei analize efectuate în 178 de companii de retail din Europa și America de Nord, sistemelor hibride PLS-SEM (Partial Least Squares Structural Equation Modeling - o metodă statistică pentru analiza relațiilor complexe dintre variabile) și a sistemelor de prognoză bazate pe machine learning a contribuit la o creștere de 3,7% a marjelor brute medii, în același timp reducând pierderile financiare legate de reducerile de preț cu aproximativ 28,6 milioane EUR anual pentru marii comercianți din retail. Aceste câștiguri de performanță provin în principal din capacitatea sporită de a detecta relații neliniare dintre variabilele pieței și modelele de comportament ale consumatorilor, pe care metodele tradiționale nu reușesc în mod constant să le identifice.²³

2.4. Limitări identificate în literatura de specialitate

2.4.1. Lipsa studiilor pe date reale din retailul românesc

²² Brownlee, J. (2017). *An Introduction to Long Short-Term Memory Networks (LSTM)*. Towards Data Science. <https://towardsdatascience.com/an-introduction-to-long-short-term-memory-networks-lstm-27af36dde85d/>

²³ <https://www.ijssat.org/papers/2025/1/2644.pdf>

Una dintre cele mai semnificative limitari descoperite este absența studiilor centrate pe piața de retail românească. Recenziile asupra aplicării machine learning în managementul stocurilor analizează predominant cazuri din piețe mature (SUA, Europa Occidentală, China). Piața românească de retail prezintă caracteristici distincte care o diferențiază de piețele occidentale: tranziția de la structuri comerciale mici și segmentate către lanțuri retail moderne a fost accelerată abia după aderarea României la Uniunea Europeană, multe companii autohtone neadaptându-se acestui ritm de schimbare [Otexts](#). În plus, provocări specifice precum preferința semnificativă pentru plata ramburs, infrastructura logistică subdezvoltată în regiunile îndepărtate și opțiunile limitate de plată online creează un context operațional distinct față de piețele dezvoltate [Exploring Economics](#).

- Lipsa studiilor aplicate pe date reale din retailul românesc
- Dificultăți de integrare a modelelor predictive în sisteme existente

Capitolul 3. Metodologia de cercetare (1–3 pagini)

3.1. Tipul cercetării

Cercetarea realizată în cadrul acestei lucrări este de tip aplicativ, bazată pe un studiu de caz și pe analiza cantitativă a datelor provenite dintr-un sistem informatic utilizat în domeniul retail. Abordarea aleasă are ca obiectiv analizarea și evaluarea utilizării metodelor predictive asupra datelor istorice de vânzări, în contextul optimizării proceselor de gestiune a stocurilor într-un sistem informatic integrat.

Studiul de caz este fundamentat pe date reale din cadrul firmei în care lucrez, extrase din baza de date operațională a unui client din retail, ceea ce permite analizarea comportamentului vânzărilor într-un cadru concret și relevant din punct de vedere organizațional. Componenta cantitativă a cercetării este justificată de natura datelor analizate, acestea fiind structurate numeric și organizate temporal, fiind adecvate aplicării metodelor statistice și de machine learning.

3.2. Datele utilizate

Setul de date utilizat în cercetare provine dintr-o bază de date relațională aferentă unui singur magazin din domeniul retail. Acesta conține informații privind vânzările produselor pe o perioadă de doi ani, începând cu anul 2024, fiind organizat la nivel de articol și timestamp zilnic. Datele includ indicatori cantitativi și valorici, necesari pentru analiza comportamentului vânzărilor și pentru aplicarea modelelor predictive.

Portofoliul de produse analizat este caracterizat de o diversitate ridicată, incluzând un număr mare de articole aparținând mai multor furnizori. Pentru a reduce complexitatea analizei și pentru a asigura relevanța rezultatelor, a fost aplicată o analiză Pareto, atât la nivel general, cât și la nivelul unui singur brand selectat. Această etapă a permis identificarea unui subset de articole reprezentative, care concentrează o parte semnificativă din volumul vânzărilor.

Principiul Pareto (cunoscut și sub numele de regula 80:20, legea câtorva factori vitali și principiul rarității factorilor) afirmă că, pentru multe rezultate, aproximativ 80% din consecințe provin din 20% din cauze („câteva cauze vitale”).²⁴

În contextul setului de date analizat în cadrul acestei lucrări, aplicarea principiului Pareto asupra vânzărilor a evidențiat faptul că un subset restrâns de articole contribuie într-o măsură semnificativă la volumul total al vânzărilor. Mai precis, analiza a arătat că aproximativ 20% dintre articolele comercializate generează o proporție dominantă din vânzările totale ale magazinului, confirmând relevanța acestui principiu în structura cererii specifice domeniului retail. Această observație a stat la baza selecției articolelor utilizate în etapele ulterioare de analiză și modelare predictivă.

3.3. Etapele cercetării

Procedura de cercetare a fost structurată în mai multe etape succesive, menite să asigure coerența procesului de analiză și reproducibilitatea rezultatelor.

În prima etapă, a fost realizată extragerea datelor din baza de date operațională, utilizând interogări SQL. Acestea au fost concepute pentru a agrega informațiile la nivel de articol și perioadă de timp, precum și pentru a identifica articolele relevante din perspectiva contribuției la vânzări.

În etapa următoare, a fost aplicată analiza Pareto, cu scopul de a selecta articolele care generează o pondere semnificativă din vânzările totale. Analiza a fost realizată inițial asupra întregului set de produse, iar ulterior restrânsă la nivelul unui singur brand, pentru a obține un set de date mai omogen și adecvat modelării predictive.

Ulterior, datele au fost prelucrate și organizate într-un format adecvat analizei, fiind eliminate înregistrările irelevante și asigurată consistența temporală a seriilor analizate. Setul de date rezultat a constituit baza pentru etapele de analiză exploratorie și de modelare predictivă.

²⁴ [Pareto principle - Wikipedia](#)

Analiza datelor a fost realizată utilizând atât metode statistice, cât și metode de machine learning, aplicate asupra seriilor temporale de vânzări. În etapa de analiză exploratorie au fost utilizate tehnici descriptive pentru identificarea distribuției vânzărilor, a sezonality și a variațiilor în timp.

Pentru prognoza cererii, au fost aplicate modele predictive asupra seriilor temporale aferente articolelor selectate prin analiza Pareto. Modelele au fost antrenate pe baza datelor istorice și evaluate în funcție de capacitatea acestora de a surprinde evoluția vânzărilor în timp. Această abordare permite compararea performanței metodelor tradiționale cu cea a modelelor bazate pe machine learning, în contextul datelor reale dintr-un sistem ERP.

3.4. Instrumente și tehnologii utilizate

Prelucrarea și analiza datelor au fost realizate utilizând limbajul de programare **Python**, datorită flexibilității acestuia și a suportului extins pentru analiza seriilor temporale și dezvoltarea modelelor predictive. Interogarea bazei de date a fost realizată prin intermediul limbajului **SQL**.

Streamlit este o bibliotecă Python open-source pentru construirea de aplicații web interactive folosind doar Python. Este ideală pentru crearea de tablouri de bord, aplicații web bazate pe date, instrumente de raportare și interfețe utilizator interactive fără a fi nevoie de HTML, CSS sau JavaScript.²⁵

3.5. Variabile și indicatori analizați

3.5.1. Variabila țintă

Variabila țintă a cercetării este reprezentată de cantitatea vândută a fiecărui articol, înregistrată la nivel temporal (zilnic). Această variabilă reflectă direct nivelul cererii și constituie principalul indicator utilizat pentru prognoza vânzărilor.

Alegerea cantității vândute ca variabilă țintă este justificată de rolul său central în procesele de gestiune a stocurilor, întrucât aceasta influențează deciziile privind reprovizionarea, dimensionarea stocurilor și prevenirea situațiilor de ruptură de stoc sau suprastocare. Variabila este utilizată sub formă de serie temporală, permițând analiza evoluției cererii în timp și aplicarea metodelor predictive.

3.5.2. Predictorii

Setul de predictorii utilizat în cadrul modelelor predictive este format din variabile derivate din structura temporală a datelor și din informațiile disponibile în baza de date.

²⁵ <https://www.geeksforgeeks.org/python/a-beginners-guide-to-streamlit/>

Principalii predictorii utilizați includ:

- Dimensiunea temporală, reprezentată prin timestamp-ul asociat fiecărei înregistrări, utilizată pentru ordonarea și structurarea seriilor temporale;
- Indicatori temporali derivați, precum luna sau perioada calendaristică, utilizați pentru surprinderea variațiilor sezoniere ale vânzărilor;
- Valori istorice ale variabilei țintă, utilizate implicit de modelele de prognoză a seriilor temporale pentru identificarea tiparelor recurente și a dependențelor în timp.

Predictorii selectați sunt adecvați scopului cercetării, întrucât permit modelarea evoluției cererii pe baza comportamentului istoric al vânzărilor, fără a introduce complexitate suplimentară prin variabile care nu sunt disponibile în mod curent în sistemele informatice de tip ERP.

Capitolul 4. Rezultate și aplicația dezvoltată (25–30 pagini)

Primul pas în dezvoltarea aplicației este reprezentat de alegerea datelor, interogarea bazelor de date pentru a obține un set optim ce este exportat sub forma de fișier csv, pentru ușurința utilizării strict în cadrul proiectului. Acest pas poate fi automatizat, creându-se script-uri python ce export automat seturile de date sau realizează funcții de merge între seturile de date cu informațiile noi adăugate și cele deja existente sau prin conexiunea directă către baza – proces ce poate fi cunoscut din punctul de vedere al operațiunilor rulate și al timpului de execuție.

Având în vedere diversitatea ridicată a articolelor disponibile la nivelul magazinului analizat, utilizarea întregului set de produse ar fi condus la o complexitate ridicată a procesului de analiză și la dificultăți în interpretarea rezultatelor.

Selecția articolelor a fost realizată prin aplicarea metodei Pareto, utilizată pentru identificarea produselor cu impact semnificativ asupra volumului total al vânzărilor.

Exemplul din Fig.1 arată o primă încercare a principiului Pareto pentru grupele de articol, ce apoi a fost realizat pentru branduri / furnizori și pentru articolele achiziționate pe un singur furnizor. Această abordare a permis obținerea unui set de date mai omogen și mai adecvat pentru analiza seriilor temporale și pentru implementarea modelelor predictive, în figura 2 este o parte din setul de date rezultat ce conține primele articole, cele mai importante din punctul de vedere al

```

select *
from (
  with baza as (
    select
      a.grupa,
      sum(v.val_iesire_fara_tva) as total_vanzari
    from f_vanzari_art_si@warehouse v
    join d_articol@warehouse a
      on a.id_articol = v.id_articol
    where v.id_gestiune = 2
    group by a.grupa
  ),
  total as (
    select sum(total_vanzari) as total_general
    from baza
  )
  select
    b.grupa,
    b.total_vanzari,
    b.total_vanzari / t.total_general as pondere,
    sum(b.total_vanzari) over (
      order by b.total_vanzari desc
    ) / t.total_general as pondere_cumulata
  from baza b
  cross join total t
  order by b.total_vanzari desc
)
where pondere_cumulata <= 0.8;

```

Fig. 1 Interogarea datelor aplicând Pareto

analizei pareto, ce vor fi folosite mai departe in lucrare, in analiza singulara si predictii pe un singur articol.

	A	D	E	F
1	ID_ARTICOL	TOTAL_VANZARI	PONDERE	PONDERE_CUMULATA
2	147807	482439.43	0.020452779	0.020452779
3	181925	469426.71	0.019901112	0.040353891
4	118033	412994.04	0.017508677	0.057862568
5	69344	399572.18	0.016939664	0.074802232
6	190760	392814.25	0.016653165	0.091455396
7	147808	377916.81	0.016021595	0.107476991
8	67303	356541.47	0.015115398	0.122592389
9	94871	287981.83	0.012208846	0.134801236
10	140755	286061.2	0.012127422	0.146928658
11	147790	278235.5	0.011795656	0.158724313
12	94869	275438.94	0.011677097	0.17040141
13	144602	272906.46	0.011569734	0.181971144
14	67074	260218.2285	0.011031822	0.193002966
15	67277	259333.06	0.010994296	0.203997262

Fig. 2 Set de date rezultat dupa pareto

Dupa alegerea articolelor, urmatorul pas a fost exportarea datelor detaliate cu privire la vanzari pentru toate articolele alese, apoi sparte pe fiecare articol.

Valorile lipsă au fost tratate diferențiat, în funcție de semnificația economică a variabilelor. Pentru cantitățile vândute, valorile lipsă au fost considerate absențe reale ale cererii și imputate cu zero. Valorile financiare asociate vânzărilor au fost imputate condiționat, doar în situațiile în care cantitatea vândută a fost zero. Valorile lipsă aferente reducerilor comerciale au fost tratate pe baza ratei de discount, pentru a evita introducerea unor distorsiuni artificiale în date.

4.1. Analiza exploratorie a datelor

Determinarea gradului de umplere a coloanelor
Calculul procentului de valori lipsă

- Determinarea liniilor cu date lipsă
Numărul liniilor care au valori lipsă

Decizii : eliminarea coloanelor sau inregistrarilor cu date lipsa

- Analiza seriilor temporale

- Identificarea sezonaliității

TIME SERIES ANALYSIS

Observațiile sunt dependente temporal

Ordinea ESTE informația

„Pentru modelele de tip serie temporală, setul de date a fost împărțit în mod cronologic, evitând amestecarea observațiilor. Utilizarea unui random split ar introduce scurgeri de informație temporală și ar conduce la evaluări nerealiste ale performanței modelului.”

4.2. Descompunerea seriilor temporale

- LOESS / trend / sezonaliitate / reziduu

```
[51]: from statsmodels.tsa.seasonal import STL

stl = STL(train, period=7, robust=True)
res = stl.fit()

res.plot()
plt.show()
```

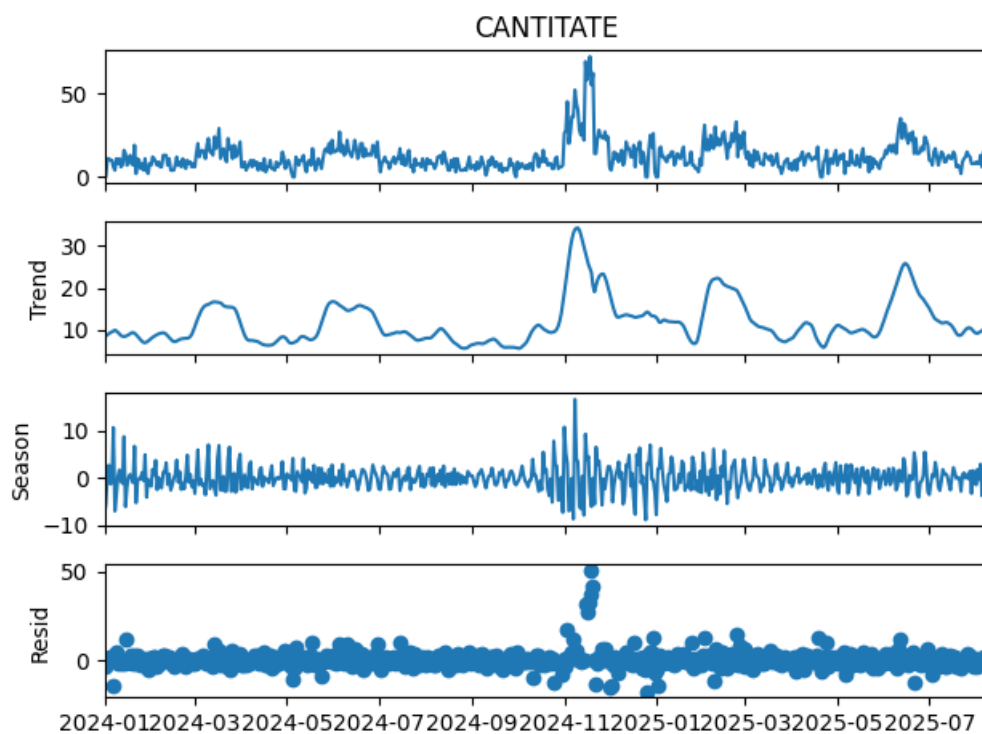


Fig. 4 STL

period = 7 $\Rightarrow$ modelul caută **sezonaliitate săptămânală**, adică tipare care se repetă **din 7 în 7 zile** (ex.: comportament diferit în weekend vs zile lucrătoare)

Observed (CANTITATE) – seria originală

Trend – evoluția lentă, pe termen mediu

Season – tiparul repetitiv săptămânal

Resid – zgomot + evenimente neexplicate

Descompunerea STL aplicată asupra seriei zilnice, utilizând o perioadă de 7 zile, evidențiază existența unei sezonality săptămânale moderate, caracterizată prin oscilații regulate de amplitudine redusă. Componenta de trend surprinde variațiile pe termen mediu ale cererii, inclusiv creșteri pronunțate în anumite perioade ale anului. Reziduurile indică prezența unor fluctuații excepționale, asociate probabil unor evenimente comerciale sau promoționale, care nu pot fi explicate prin structura sezonieră săptămânală.

4.3. Modele de prognoză utilizate

ARIMA

```
[20]: # import libraries
      from statsmodels.tsa.stattools import adfuller

      # ADF test
      result = adfuller(train)
      print('ADF Statistic:', result[0])
      print('p-value:', result[1])

      # if p-value > 0.05, the series is non-stationary and needs differencing
      if result[1] > 0.05:
          print("needs differencing")
      else:
          print("time series is stationary")

      ADF Statistic: -3.7874346810000405
      p-value: 0.003038522956429229
      time series is stationary
```

Staționaritatea seriei temporale a cantității vândute a fost evaluată utilizând testul Augmented Dickey-Fuller. Rezultatul obținut indică o valoare p de 0,003, sub pragul de semnificație de 0,05, ceea ce conduce la respingerea ipotezei nule și confirmă caracterul staționar al seriei. În consecință, pentru modelul ARIMA nu a fost necesară aplicarea diferențierii, parametrul d fiind stabilit la valoarea zero.

O serie temporală staționară este un tip de serie temporală ale cărei proprietăți statistice nu se modifică în timp. Aceasta înseamnă că comportamentul statistic al procesului rămâne constant, indiferent de momentul în care au fost înregistrate observațiile.²⁶ Seria are

²⁶ <https://www.tigerdata.com/learn/stationary-time-series-analysis>

o **medie constantă pe termen lung** și nu are un "unit root" (nu crește sau scade la infinit, ca un trend liniar).

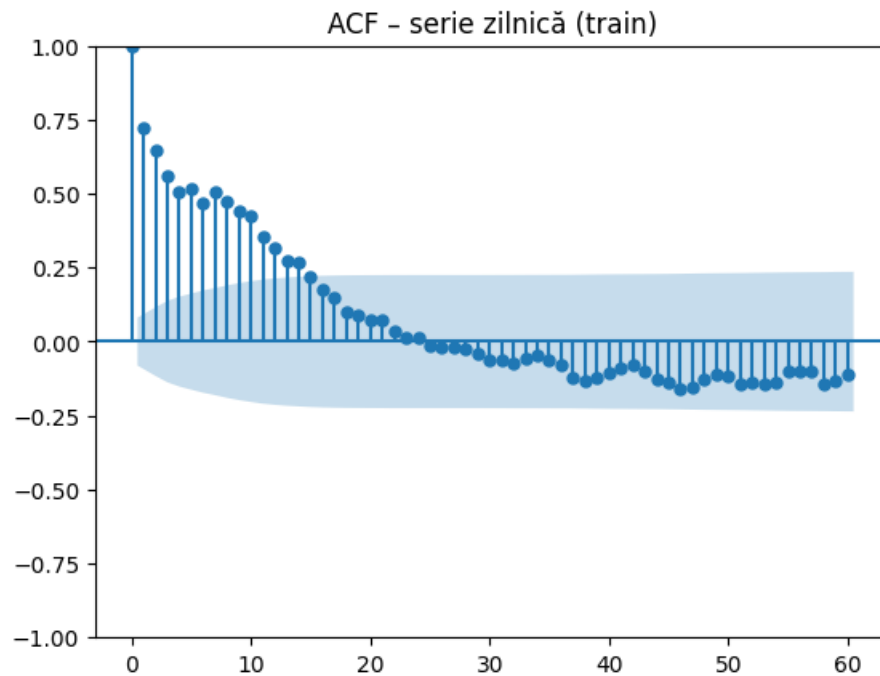
Tendențele care se repetă pe parcursul zilelor sau lunilor se numesc sezonabilitate în seriile temporale. Schimbările sezoniere, festivalurile și evenimentele culturale produc adesea aceste variații.²⁷

Nu facem diferențiere $\rightarrow d = 0$.

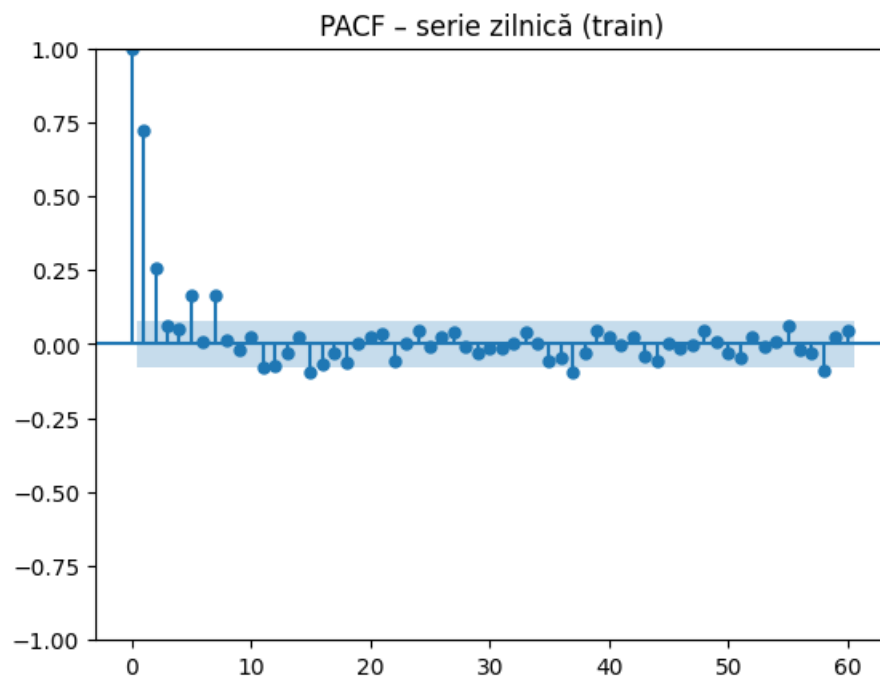
Model	P / "AR"	Q / "MA"	MAE	RMSE	R ²
ARIMA	5	4	7,02	11,1	0.0009

Rezultatele obținute pentru modelul ARIMA indică o capacitate limitată de prognoză, reflectată prin valori mai ridicate ale erorilor MAE și RMSE, precum și printr-un coeficient de determinare apropiat de zero. Acest rezultat sugerează că modelul reușește să surprindă nivelul mediu al cererii, însă nu explică în mod satisfăcător variațiile acesteia în timp.

²⁷ <https://www.analyticsvidhya.com/blog/2024/06/seasonality-in-time-series/>

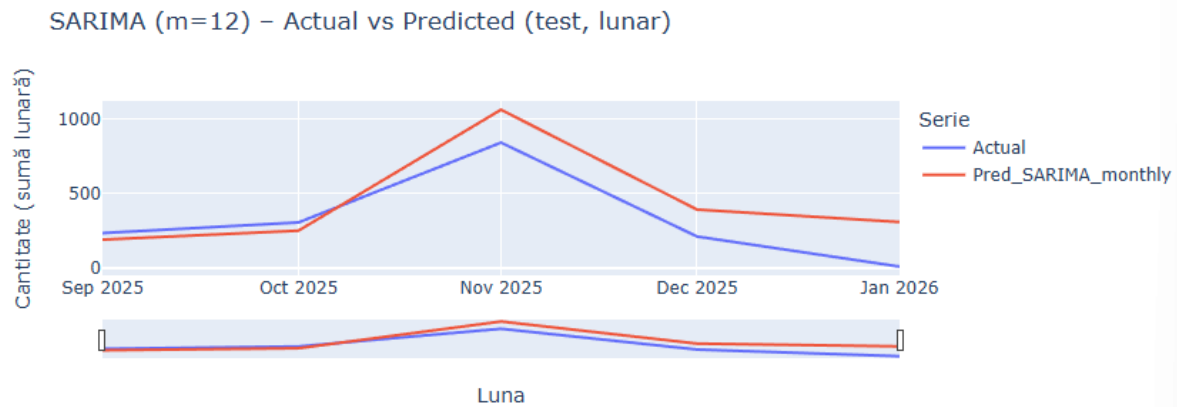


<Figure size 1000x400 with 0 Axes>



Analiza funcțiilor de autocorelație (ACF) și autocorelație parțială (PACF) pentru seria zilnică indică o dependență puternică de valorile recente, cu o scădere treptată a autocorelației și un număr redus de lag-uri semnificative în PACF. Acest comportament este caracteristic unui proces autoregresiv de ordin redus, sugerând utilizarea unui model ARIMA cu parametru p egal cu 1 sau 2. În același timp, lipsa unor vârfuri periodice în ACF indică absența unei sezonality zilnice pronunțate, deși analiza vizuală evidențiază variații recurente la nivel anual, asociate anumitor luni calendaristice.

SARIMA



Pentru a investiga existența unei sezonality anuale în evoluția cererii, seria zilnică a fost agregată la nivel lunar, iar ulterior a fost estimat un model SARIMA cu periodicitate anuală ($m = 12$). Alegerea parametrilor a fost realizată automat, utilizând criteriul informațional AIC. Practic, modelul se bazează exclusiv pe variațiile dintre aceeași lună din ani diferiți, fără a utiliza informație suplimentară din observațiile recente.

Model	m	Q / "MA"	MAE	RMSE	R ²
ARIMA	5	4	7,02	11,1	0.0009

Analiza performanței pe setul de test evidențiază valori relativ ridicate ale erorilor ($MAE \approx 159$, $RMSE \approx 187$), în timp ce coeficientul de determinare indică o capacitate moderată de explicare a variației cererii ($R^2 \approx 0,55$). Graficul comparativ „Actual vs. Predicted” arată că modelul reușește să surprindă direcția generală a evoluției cererii, inclusiv creșterea din luna noiembrie, însă tinde să supraestimeze amplitudinea vârfurilor și să subestimeze scăderile ulterioare. (

Aceste rezultate sugerează că variațiile observate în lunile cu vânzări ridicate nu reprezintă o sezonality anuală stabilă, ci mai degrabă efecte calendaristice sau promoționale, cu intensitate variabilă de la un an la altul. În consecință, modelul SARIMA lunar are o utilitate limitată pentru prognoza precisă a cererii și este adecvat în principal ca instrument exploratoriu, pentru evidențierea existenței (sau inexistenței) unui tipar sezonier anual constant.

LSTM

Pentru prognoza cererii a fost utilizat un model de tip Long Short-Term Memory (LSTM), aparținând clasei rețelelor neuronale recurente, adecvat pentru modelarea seriilor temporale caracterizate prin dependențe temporale. Spre deosebire de modelele de tip machine learning clasic, în care observațiile sunt considerate independente, modelul LSTM permite captarea relațiilor dintre valorile succesive ale unei serii temporale.

Train Test Split

În învățarea automată, este o practică obișnuită să se împartă datele în două seturi diferite. Aceste două seturi sunt **setul de antrenament** și **setul de testare**. După cum sugerează și numele, setul de antrenament este utilizat pentru antrenarea modelului, iar setul de testare este utilizat pentru testarea acurateții modelului.

Cel mai comun raport de divizare este **80:20**. Adică 80% din setul de date intră în setul de antrenament, iar 20% din setul de date intră în setul de testare.²⁸

Pentru a evita parametrul `random_state` din funcția automată `train_test_split`, care implică o amestecare aleatorie a observațiilor și poate provoca pierderea ordinii cronologice în seriile temporale, împărțirea setului de date a fost realizată în mod secvențial, pe baza timpului. (Fig. 1)

```
[55]: split_ratio = 0.8
      n = len(df_lstm)
      split_idx = int(n * split_ratio)

      split_idx

[55]: 587

[56]: train = df_lstm.iloc[:split_idx]
      test = df_lstm.iloc[split_idx:]

[57]: # Scaling - doar pe TRAIN TEST

[58]: scaler = MinMaxScaler()
      train_scaled = scaler.fit_transform(train[["CANTITATE"]])
      test_scaled = scaler.transform(test[["CANTITATE"]])
```

Fig. 6 Impartire si standardizare a datelor

Am continuat prin scalarea datelor. Scalarea este, procesul de transformare a tuturor caracteristicilor dintr-un model mai aproape de un interval sau pe aceeași scară, de exemplu: toate valorile să fie între 0 și 1. De asemenea, anumiți algoritmi, converg mai bine atunci când caracteristicile sunt scalate, deci performanța va fi îmbunătățită.

Având în vedere faptul că datele analizate sunt de tip serie temporală, setul de caracteristici utilizat în modelare nu este compus preponderent din variabile independente distincte, ci din aceeași variabilă principală, respectiv cantitatea vândută, deplasată în timp sub forma unor ferestre temporale. Astfel, scalarea a fost aplicată pentru a asigura consistența numerică a valorilor istorice utilizate în procesul de învățare al modelului.

Am folosit scalarea min-max (`MinMaxScaler`) – denumită și normalizare – ce transformă datele astfel încât fiecare valoare să se încadreze între 0 și 1, ce funcționează conform formulei din Ecuația 1. (Eq. 1).²⁹

$$X(\text{scaled}) = (X_i - X_{\min}) / (X_{\max} - X_{\min})$$

Eq. 1 Ecuația de normalizare pentru un punct de date X_i

²⁸ <https://www.askpython.com/python/examples/split-data-training-and-testing-set>

²⁹ <https://towardsdatascience.com/data-scaling-101-standardization-and-min-max-scaling-explained-60789833e160/>

În cadrul acestui studiu, predicția cantității vândute la momentul *t* se realizează pe baza unei ferestre temporale de observații anterioare, denumită *lookback window*. Aceasta reprezintă numărul de pași temporali din trecut utilizați ca intrare în model. În implementarea curentă, dimensiunea ferestrei a fost stabilită la 14 zile, valoare aleasă pentru a surprinde atât variațiile pe termen scurt, cât și eventualele pattern-uri săptămânale ale cererii.

Rolul datasetului istoric extins

Deși setul de date analizat acoperă o perioadă de aproximativ doi ani, acesta nu este utilizat integral pentru o singură predicție. Istoricul extins servește la generarea unui număr mare de secvențe de antrenare, fiecare secvență fiind construită prin deplasarea ferestrei temporale de tip *lookback* de-a lungul seriei. Acest mecanism permite modelului să învețe tipare recurente ale cererii în contexte temporale diferite, menținând în același timp o structură de predicție realistă din punct de vedere operațional.

Rezultat:

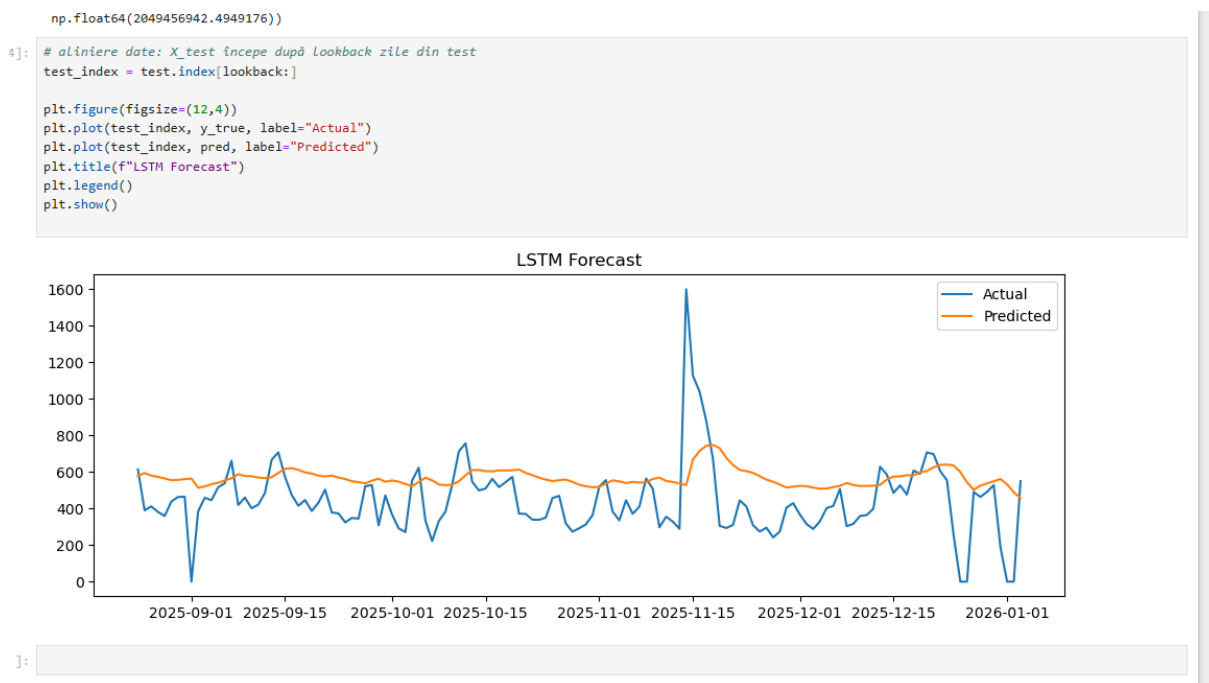


Fig. 7 LSTM Predicted

Pe scurt, rezultatul cuprinde valorile medii precise, valori stabile, utilizabile.

Graficul comparativ dintre valorile reale și cele estimate de modelul LSTM evidențiază capacitatea acestuia de a surprinde nivelul general și tendința cererii, reducând în același timp impactul fluctuațiilor extreme izolate. Predicțiile obținute sunt mai netede decât seria observată, ceea ce indică faptul că modelul filtrează zgomotul și evenimentele atipice, concentrându-se pe comportamentul recurent al cererii. Această caracteristică este avantajoasă în contextul planificării stocurilor, unde estimarea cererii medii este mai relevantă decât replicarea exactă a variațiilor zilnice extreme. (aici tb sa reformulezi, Mara)

```
[63]: from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(y_true, pred)

mse = mean_squared_error(y_true, pred)
rmse = np.sqrt(mse)

mape = np.mean(np.abs((y_true - pred) / np.maximum(y_true, 1))) * 100

mae = float(mae)
rmse = float(rmse)
mape = float(mape)

mae, rmse, mape
```

```
[63]: (5.25101833873325, 8.066700162880037, 83.71565125019796)
```

```
[64]: from sklearn.metrics import r2_score

r2 = r2_score(y_true, pred)
r2
```

```
[64]: 0.5388633412727699
```

```
[65]: import numpy as np

r = np.corrcoef(y_true, pred)[0, 1]
float(r)
# arata daca modelul urmareste trandul, nu cat de exact prezice
```

```
[65]: 0.7346987072878048
```

Analiza performanței modelului LSTM (lookback = 21)

Modelul LSTM multivariant, construit pe baza unei ferestre temporale de 21 de zile, a fost evaluat utilizând setul de testare, prin intermediul unor metrici specifice problemelor de regresie aplicate seriilor temporale.

Valoarea **MAE**, de **5,25 unități**, arată că, în medie, diferența dintre cantitatea estimată de model și cea observată în realitate este de aproximativ cinci unități pe zi. Raportat la nivelul general al cererii și la variabilitatea caracteristică datelor din retail, această eroare poate fi considerată rezonabilă, indicând o acuratețe suficientă pentru utilizarea modelului în procese de planificare a stocurilor.

În cazul **RMSE**, valoarea obținută este de **8,07 unități**, mai ridicată decât MAE, ceea ce evidențiază faptul că modelul înregistrează erori mai mari în situațiile în care apar variații extreme ale cererii. Acest rezultat este așteptat în contextul seriilor temporale provenite din sisteme ERP, unde pot apărea vârfuri izolate sau fluctuații bruște, pe care modelul LSTM tinde să nu le reproducă exact.

Coefficientul de determinare R^2 , cu valoarea de **0,54**, indică faptul că modelul explică aproximativ jumătate din variația cantității vândute. Pentru o serie temporală reală, influențată de factori externi și caracterizată printr-un nivel ridicat de zgomot, acest rezultat sugerează o capacitate bună de surprindere a comportamentului general al cererii.

De asemenea, **coeficientul de corelație Pearson**, egal cu **0,73**, confirmă existența unei relații pozitive puternice între valorile reale și cele estimate. Acest lucru arată că modelul reușește

să urmărească direcția și evoluția generală a cererii, chiar dacă nu redă cu precizie toate variațiile zilnice.

În ceea ce privește **MAPE**, valoarea ridicată obținută (**83,7%**) este influențată de prezența valorilor nule sau foarte mici în seria analizată, situație care conduce la supraestimarea erorii procentuale. Din acest motiv, această metrică nu a fost considerată relevantă pentru evaluarea principală a performanței modelului.³⁰

Concluzie

În ansamblu, modelul LSTM multivariant cu o fereastră de 21 de zile oferă rezultate satisfăcătoare pentru estimarea nivelului general al cererii și pentru identificarea tendințelor dominante. Deși nu surprinde cu exactitate fluctuațiile extreme sau evenimentele izolate, caracterul său stabil și conservator îl face adecvat pentru susținerea deciziilor operaționale legate de gestiunea și optimizarea stocurilor.

LSTM 14

```
[57]: from sklearn.metrics import mean_absolute_error, mean_squared_error
import numpy as np

mae = mean_absolute_error(y_true, pred)

mse = mean_squared_error(y_true, pred)
rmse = np.sqrt(mse)

mape = np.mean(np.abs((y_true - pred) / np.maximum(y_true, 1))) * 100

mae = float(mae)
rmse = float(rmse)
mape = float(mape)

mae, rmse, mape
```

```
[57]: (5.158669378524436, 8.204768418153085, 79.19503815748314)
```

```
[58]: from sklearn.metrics import r2_score

r2 = r2_score(y_true, pred)
r2
```

```
[58]: 0.5011708076545767
```

```
[59]: import numpy as np

r = np.corrcoef(y_true, pred)[0, 1]
float(r)
# arata daca modelul urmareste trandul, nu cat de exact prezice
```

```
[59]: 0.7111825687299554
```

³⁰ <https://www.geeksforgeeks.org/machine-learning/regression-metrics/>

Model	Lookback (zile)	MAE	RMSE	R ²	r (Pearson)
LSTM	14	5,16	8,20	0,50	0,71
LSTM	21	5,25	8,07	0,54	0,73

Tabel 1. Compararea performanței modelului LSTM în funcție de dimensiunea ferestrei temporale

Deși modelul LSTM cu o fereastră temporală de 14 zile obține o eroare medie absolută ușor mai redusă, diferența față de modelul cu lookback de 21 de zile este nesemnificativă. În schimb, modelul cu 21 de zile înregistrează valori mai bune pentru RMSE, coeficientul de determinare și corelația Pearson, indicând o capacitate superioară de a surprinde structura temporală și tendința generală a cererii. Prin urmare, modelul LSTM cu lookback de 21 de zile a fost ales ca variantă finală pentru analiza ulterioară.

Stl , one step ahead, arima sarima

- Model ARIMA
- Alte modele (regresie, XGBoost, LSTM – dacă le aplici)

4.4. Evaluarea performanței modelelor

- Metrice utilizate
- Compararea rezultatelor

Pentru toti algoritmi ar trebui sa am performance tuning (sa modific aparmetrii : train test , lookback , pdq etc ...) pentru a gasi cea mai concludenta posibilitate a modelului .

4.5. Aplicația de tip dashboard pentru vizualizare

Python Streamlit

Aplicația este dezvoltată utilizând framework-ul Streamlit, care pornește un server web local pe stația de lucru, expunând aplicația prin intermediul unui port TCP dedicat (implicit portul 8501), fiind accesibilă fie direct de pe stația pe care rulează aplicația (<http://localhost:8501>) , fie prin intermediul adresei IP din rețeaua internă (Network URL), de exemplu <http://192.168.1.128:8501>.

Acest mod de acces este specific mediilor organizaționale care utilizează infrastructuri proprii de tip *on-premise* sau centre de date interne, în care resursele informatice sunt partajate în

cadrul unei rețele locale securizate (LAN), fără expunerea aplicației în mediul public (Internet).

Accesarea aplicației este limitată la utilizatorii aflați în aceeași rețea internă, ceea ce permite controlul asupra securității datelor și respectarea politicilor interne de confidențialitate, fiind o abordare frecvent utilizată în cadrul organizațiilor care gestionează date sensibile.

Aplicația a fost dezvoltată utilizând un **mediu virtual Python (Python Virtual Environment)**, ales pentru a asigura un control riguros asupra dependențelor software utilizate în cadrul proiectului. Acest mediu izolat permite rularea aplicației în condiții identice pe sisteme diferite, prin utilizarea acelorași versiuni ale bibliotecilor necesare, conform specificațiilor din fișierul *requirements.txt*.

Astfel, aplicația poate fi rulată în aceleași condiții software pe orice sistem, fără diferențe de versiuni ale pachetelor utilizate. mecanism oferă **izolație față de alte proiecte Python** existente pe sistemul de operare, prevenind conflictele între biblioteci sau modificările neintenționate ale mediului global.³¹

- Descriere generală a aplicației
- Indicatori afișați
- Utilitate pentru decizie managerială

Pentru dezvoltarea aplicației software și refactorizarea codului Python au fost utilizate instrumente moderne de asistare a programării, precum **GitHub Copilot** (integrat în Visual Studio Code) și platforma **Replit**, care au sprijinit procesul de generare și organizare a codului.

Logica de analiză a datelor, definirea modelelor predictive și interpretarea rezultatelor au fost realizate și validate de autor.

4.6. Automatizarea fluxului de date (opțional – n8n)

- Rolul automatizării

³¹ <https://docs.python.org/3/library/venv.html>

- Beneficii operaționale

„Pentru a facilita utilizarea practică a rezultatelor obținute, a fost dezvoltată o interfață de tip dashboard, care permite vizualizarea evoluției stocurilor și a prognozelor generate.”

„Pentru automatizarea procesului de actualizare a datelor și rulare a modelelor predictive, a fost implementat un flux de orchestrare a proceselor.”

React + n8n = **10–15% din lucrare**, maxim

ML + analiză = **focus principal**

Capitolul 5. Discuții și analize (*aprox. 10 pagini*)

5.1. Interpretarea rezultatelor

- Impact asupra gestiunii stocurilor
- Compararea cu rezultatele din literatura de specialitate

5.2. Implicații pentru compania analizată

- Decizii de aprovizionare
- Optimizarea stocului tampon

5.3. Implicații pentru domeniul sistemelor informatice integrate

CAPITOLUL 6 – CONCLUZII, PROPUNERI ȘI IMPLICAȚII PRACTICE

Limitări, concluzii și recomandări practice (3–5 pagini)

6.1. Limitările studiului

- Date
- Perioadă
- Generalizare

6.2. Concluziile principale

- Răspuns la obiectivele cercetării

6.3. Recomandări practice

- Pentru companie

În locul utilizării unui mediu virtual Python configurat local, aplicația poate fi încapsulată sub forma unui container Docker, care include atât codul aplicației, cât și toate dependențele necesare. Această abordare permite partajarea aplicației și rularea sa într-un mod consistent de către membrii echipei interne a organizației, indiferent de mediul local de execuție.

Pentru implementări viitoare

6.4. Direcții viitoare de cercetare

Z.1. Concluzii

Z.2. Propuneri

Z.3. Implicații practice

ce ai demonstrat

utilitatea practică

limitări

ce se poate extinde (mai multe produse, date zilnice, integrare ERP)

 Ordinea corectă este:

Retail → problemă → date → data science → soluție

BIBLIOGRAFIE